FinSight – Complete Test Cases Document

Covers all unit, integration, property-based, and controller (MockMvc) tests across the backend.

1. Fraud Detection Service – Unit Tests

File: FraudDetectionServiceTest.java | Framework: JUnit 5 + Mockito

ID	Test Method	Scenario	Setup / Mocks	Expected
UT-FD-01	analyzeTransaction_HighAmountAnomaly_Adds30Points	Transaction is $400; user average is $100 (>3x average)	avg=$100; 0 rapid-fire txns; no prior location; known category 'groceries'	score ≥ 30.0; reason contains 'exceeds 3x user average'
UT-FD-02	analyzeTransaction_RapidFireActivity_Adds25Points	5+ transactions recorded for the same user in the last 10 minutes	avg=$100; rapid-fire count=5; no prior location; known category	score ≥ 25.0; reason = '5 or more transactions within 10 minutes'
UT-FD-03	analyzeTransaction_GeographicalAnomaly_Adds25Points	Last tx in 'New York' 1 hour ago; current tx from 'Los Angeles'	Last tx: location=New York, 1 hr ago; current: location=Los Angeles	score ≥ 25.0; reason = 'Different location within 2 hours of previous transaction'
UT-FD-04	analyzeTransaction_UnusualCategory_Adds20Points	User known categories are ['groceries','utilities']; tx category = 'entertainment'	avg=$100; 0 rapid-fire; no prior location; known categories=['groceries','utilities']	score ≥ 20.0; reason = 'First time using category: entertainment'
UT-FD-05	analyzeTransaction_NoFraudIndicators_ReturnsZeroScore	Normal transaction — amount within average, known category, no anomalies	avg=$100; 0 rapid-fire; no prior location; known category 'groceries'; amount=$50	score = 0.0; isFraudulent = false; riskLevel = LOW
UT-FD-06	analyzeTransaction_HighScore_SetsFraudulentFlag	Multiple rules triggered: high amount + rapid-fire + new category	avg=$100; rapid-fire=5; amount=$400; new category 'entertainment'	score ≥ 70.0; isFraudulent = true; riskLevel = HIGH

2. Fraud Detection Service – Integration Tests

File: FraudDetectionIntegrationTest.java | Framework: JUnit 5 + Mockito (@Mock lenient)

ID	Test Method	Scenario	Setup / Mocks	Expected
IT-FD-01	multipleFraudIndicators_AccumulateScores	High amount + rapid-fire + new category all triggered at once	avg=$100; rapid-fire=5; amount=$400; new category 'crypto'	score ≥ 70.0; isFraudulent=true; riskLevel=HIGH; reasons.size=3
IT-FD-02	extremeAmount_TriggersHighFraudScore	Transaction is $10,000 against avg $100 (100× average)	avg=$100; 0 rapid-fire; no prior location; known categories	score ≥ 50.0; reason contains 'exceeds 3x user average'
IT-FD-03	geographicalAnomalyWithRapidTransaction_Flagged	Last tx in 'New York' 30 min ago; current tx from 'Tokyo'	Last tx: location=New York, 30 min ago; current: location=Tokyo	score ≥ 25.0; reason = 'Different location within 2 hours of previous transaction'
IT-FD-04	sameLocation_NoGeographicalAnomaly	Both transactions from 'New York'	Last tx: location=New York, 30 min ago; current: location=New York	reasons do NOT contain 'Different location within 2 hours'
IT-FD-05	knownCategory_NoNewCategoryAlert	Transaction uses a category the user has used before	known categories=['groceries','utilities']; tx category='groceries'	reasons do NOT contain 'New category for user'
IT-FD-06	riskLevel_LowForScoreBelow40	Normal transaction — no fraud indicators	avg=$100; 0 rapid-fire; known category; amount=$50	riskLevel = LOW; score < 40.0
IT-FD-07	riskLevel_MediumForScoreBetween40And70	Two rules triggered: high amount (30 pts) + new category (20 pts) = 50 pts	avg=$100; amount=$350 (>3x); 0 rapid-fire; new category 'entertainment'	riskLevel = MEDIUM; score between 40.0 and 70.0
IT-FD-08	riskLevel_HighForScore70OrAbove	High amount + rapid-fire (6	avg=$100; rapid-fire=6; amount=$500; new category 'crypto'	riskLevel = HIGH; score ≥ 70.0

ID	Test Method	Scenario	Setup / Mocks	Expected
		txns) + new category = HIGH		
IT-FD-09	rapidFire_10Plus_Adds40Points	10+ transactions in the last 10 minutes	rapid-fire count=10; avg=$100; known category	score ≥ 25.0; reason = '5 or more transactions within 10 minutes'
IT-FD-10	firstTransaction_NoErrors	User's very first transaction — no historical data	calculateAverageAmount returns null; 0 rapid-fire; empty tx list; empty categories	result is not null; score ≥ 0.0; no exception thrown
IT-FD-11	zeroAmount_NoErrors	Transaction amount is exactly $0.00	avg=$100; 0 rapid-fire; known category; amount=0	result is not null; score ≥ 0.0; no exception thrown

3. Fraud Detection Service – Property-Based Tests

File: FraudDetectionServicePropertyTest.java | Framework: jqwik (100 tries each)

ID	Method	Property	Tries	Generators	Invariant
PT-FD-01	fraudScoreIsAlwaysBetween0And100	Fraud Score Bounds	100	Arbitrary transactions: random id (1–10000), amount (1–10000), type (INCOME/EXPENSE), category	result.fraudScore is between 0.0 and 100.0 (inclusive)
PT-FD-02	fraudulentFlagSetCorrectlyBasedOnScore	Fraud Score Threshold Enforcement	100	Arbitrary doubles in [0.0, 100.0]	score ≥ 70 → expectedFraudulent=true; score < 70 → false
PT-FD-03	riskLevelMappedCorrectly	Risk Level Mapping Correctness	100	Arbitrary doubles in [0.0, 100.0]	score < 40 → LOW; 40 ≤ score < 70 → MEDIUM; score ≥ 70 → HIGH
PT-FD-04	fraudDetectionIsDeterministic	Fraud Detection Determinism	100	Arbitrary transactions; fixed mocks (avg=100, 0 rapid-fire, no location)	result1 and result2 identical: same score, same isFraudulent, same riskLevel

4. Fraud Alert Service – Unit Tests

File: FraudAlertServiceTest.java | Framework: JUnit 5 + Mockito

ID	Method	Scenario	Setup	Expected
UT-FA-01	findByUser_NoFilters_ReturnsAllAlerts	No resolved/severity filter applied	User has 2 alerts (HIGH unresolved, MEDIUM unresolved); no filters	Returns list of 2 alerts; messages match 'Suspicious transaction detected' and 'Another alert'
UT-FA-02	findByUser_WithResolvedFilter_ReturnsFilteredAlerts	Filter by resolved=false	1 unresolved alert; resolved filter=false	Returns 1 alert; isResolved=false; message='Unresolved alert'
UT-FA-03	findByUser_WithSeverityFilter_ReturnsFilteredAlerts	Filter by severity='HIGH'	1 HIGH alert; severityFilter='HIGH'	Returns 1 alert; severity='HIGH'
UT-FA-04	findByUser_WithBothFilters_ReturnsFilteredAlerts	Filter by resolved=false AND severity='HIGH'	1 HIGH unresolved alert; both filters applied	Returns 1 alert; isResolved=false; severity='HIGH'
UT-FA-05	findByUser_UserNotFound_ThrowsException	userId=999 does not exist	userRepository.findById(999) returns empty	RuntimeException thrown; message contains 'User not found'
UT-FA-06	resolveAlert_ExistingAlert_ResolvesSuccessfully	Valid alertId and userId; alert exists and belongs to user	fraudAlertRepository.findById(1) returns testAlert; save returns testAlert	Returns non-null DTO; save() called once; testAlert.isResolved=true
UT-FA-07	resolveAlert_AlertNotFound_ThrowsException	alertId=999 does not exist	fraudAlertRepository.findById(999) returns empty	RuntimeException thrown; message contains 'Fraud alert not found'
UT-FA-08	resolveAlert_UnauthorizedUser_ThrowsException	Alert exists but belongs to userId=1; caller is userId=999	fraudAlertRepository.findById(1) returns testAlert (owned by user 1); called with userId=999	RuntimeException thrown; message contains 'Unauthorized access'
UT-FA-09	resolveAlert_AlreadyResolved_StillSucceeds	Alert is already resolved; resolve called again	testAlert.resolved=true; findById returns it; save returns it	Returns DTO with isResolved=true; save() still called once
UT-FA-10	toDto_MapsCorrectly	DTO mapping verification after resolveAlert	testAlert: id=1, user=testUser, message='Suspicious transaction detected', severity=HIGH	dto.id=1; dto.userId=1; dto.message='Suspicious transaction detected'; dto.severity='HIGH'; dto.transaction.id=1

ID	Method	Scenario	Setup	Expected
UT-FA-11	findByUser_ReturnsMultipleAlerts	User has 3 alerts of different severities	3 alerts: HIGH, MEDIUM, LOW (all unresolved); no filters	Returns 3 alerts; first alert id=1

5. Transaction Service – Unit Tests

File: TransactionServiceTest.java | Framework: JUnit 5 + Mockito

ID	Method	Scenario	Setup	Expected
UT-TS-01	createTransaction_ValidRequest_CreatesAndReturnTransaction	Valid transaction request with userId=1, amount=$100, category='groceries'	user found; fraud result: score=0, LOW; transactionRepository.save returns testTransaction	response not null; amount=$100; category='groceries'; isFraudulent=false; save() called twice; analyzeTransaction() called once
UT-TS-02	createTransaction_FraudDetected_MarksFraudulent	Transaction triggers fraud detection (score=85, HIGH)	fraudResult: fraudulent=true; score=85.0; riskLevel=HIGH; reasons=['High amount anomaly']	response.isFraudulent=true; response.fraudScore=85.0; saved transaction has fraudulent=true and fraudScore=85.0
UT-TS-03	createTransaction_UserNotFound_ThrowsException	userId=999 does not exist	userRepository.findById(999) returns empty	RuntimeException thrown; message contains 'User not found'
UT-TS-04	findWithFilters_ReturnsFilteredTransactions	Filter by type='EXPENSE', category='groceries', page 0, size 10	user found; transactionRepository.findAll returns mocked page	Page result is not null; findAll(Specification, Pageable) called once
UT-TS-05	findWithFilters_UserNotFound_ThrowsException	findWithFilters called with userId=999	userRepository.findById(999) returns empty	RuntimeException thrown; message contains 'User not found'
UT-TS-06	createTransaction_SetsTransactionDateToNow	Transaction created — createdAt should be populated	valid request; user found; fraud result: LOW	saved transaction has non-null createdAt field
UT-TS-07	createTransaction_HighFraudScore_CreatesFraudAlert	Transaction with fraud score=85 triggers a FraudAlert record	fraudResult: fraudulent=true; score=85.0; HIGH	fraudAlertRepository.save() called at least once with a FraudAlert

6. Demo Data Service – Unit Tests

File: DemoDataServiceTest.java | Framework: JUnit 5 + Mockito

ID	Method	Scenario	Setup	Expected
UT-DD-01	seedUserIfEmpty_NoExistingTransactions_CreatesTransactions	User has 0 existing transactions — seeding should create them	transactionCount=0; fraudResult: LOW, score=0; save assigns id=1	returns count > 0; save() called ≥1 time; analyzeTransaction() called ≥1 time
UT-DD-02	seedUserIfEmpty_ExistingTransactions_SkipsSeeding	User already has 10 transactions — seeding should be skipped	transactionCount=10	returns 0; save() never called
UT-DD-03	seedUserIfEmpty_UserNotFound_ThrowsException	userId=999 does not exist	userRepository.findById(999) returns empty	RuntimeException thrown; message contains 'User not found'
UT-DD-04	forceReseedUser_DeletesAndCreatesNew	Force reseed: deletes existing alerts and transactions, then creates new ones	1 existing alert, 1 existing transaction; save assigns new ids	returns count > 0; fraudAlertRepository.deleteAll() called; transactionRepository.deleteAll() called; save() called ≥1 time
UT-DD-05	demoTransactions_SpanMultipleMonths	Generated transactions should span more than one calendar month	transactionCount=0; all fraud results: LOW	captured transactions not empty; distinct months > 1
UT-DD-06	demoTransactions_IncludeFraudScenarios	Some generated transactions should have amounts > $1,000 (fraud triggers)	transactionCount=0; fraud result is HIGH for amount > $1,000	captured transactions not empty; at least 1 transaction with amount > $1,000
UT-DD-07	fraudAlerts_CreatedForFraudulentTransactions	Fraudulent transactions (score ≥ 40) should produce	all transactions return fraudResult: fraudulent=true, score=85, HIGH	fraudAlertRepository.save() called ≥1 time

ID	Method	Scenario	Setup	Expected
		FraudAlert records		
UT-DD-08	transactions_HaveValidCategories	All generated transactions must have a non-null, non-empty category	transactionCount=0; all fraud results: LOW	all captured transactions have non-null and non-empty category
UT-DD-09	transactions_HaveValidAmounts	All generated transactions must have a non-null, non-negative amount	transactionCount=0; all fraud results: LOW	all captured transactions have non-null amount ≥ 0
UT-DD-10	generation_IsDeterministic	Two consecutive calls for the same userId should produce the same count	transactionCount=0 (both runs); same userId=1	count1 == count2; firstRun.size == secondRun.size

7. Auth Service – Unit Tests

File: AuthServiceTest.java | Framework: JUnit 5 + Mockito (LENIENT strictness)

7a. signup() tests

ID	Method	Scenario	Setup	Expected
UT-AS-01	signup_success_completeResponse	Valid new user signup	username/email not taken; encode returns HASH; save returns savedUser; JWT generated	response: userId=1; username='alice'; email='alice@example.com'; token=JWT; message contains 'success'
UT-AS-02	signup_passwordEncoded	Password must be BCrypt-encoded before persistence	stubSignup()	saved user's password equals HASH; does not contain raw password
UT-AS-03	signup_encodeCalledOnce	encode() should be invoked exactly once with the raw password	stubSignup()	passwordEncoder.encode(RAW) called exactly once

ID	Method	Scenario	Setup	Expected
UT-AS-04	signup_userRoleAssigned	Saved user should have 'USER' role	stubSignup()	saved user roles contain exactly 'USER'
UT-AS-05	signup_createdAtSet	createdAt should be populated on the persisted user	stubSignup()	saved user's createdAt is after test start time
UT-AS-06	signup_jwtUsesPersistedIds	JWT must use the userId and username from the persisted entity	stubSignup()	jwtUtil.generateToken(1L, 'alice') called
UT-AS-07	signup_demoDataAfterSave	Demo data seeding must happen strictly after user save()	stubSignup()	InOrder: userRepository.save() before demoDataService.seedUserIfEmpty(1L)
UT-AS-08	signup_savesOnce	userRepository.save() should be called exactly once	stubSignup()	userRepository.save() called exactly 1 time
UT-AS-09	signup_duplicateUsername_throws	Username 'alice' already exists	findByUsername('alice') returns existing user	RuntimeException thrown; message contains 'Username already exists'
UT-AS-10	signup_duplicateUsername_noSave	Duplicate username — no save should occur	findByUsername('alice') returns existing user	userRepository.save() never called
UT-AS-11	signup_duplicateEmail_throws	Email 'alice@example.com' already registered	findByUsername empty; findByEmail returns existing user	RuntimeException thrown; message contains 'Email already exists'
UT-AS-12	signup_duplicateEmail_noSave	Duplicate email — no save should occur	findByUsername empty; findByEmail returns existing user	userRepository.save() never called
UT-AS-13	signup_usernameCheckedFirst	Username is taken — email check should be skipped	findByUsername('alice') returns existing user	Exception message contains 'Username already exists'; findByEmail never called
UT-AS-14	signup_demoDataThrows_signupStillSucceeds	DemoDataService throws — signup should still complete	stubSignup(); seedUserIfEmpty throws RuntimeException('DB down')	No exception propagated from signup()
UT-AS-15	signup_demoDataThrows_tokenStillReturned	DemoDataService throws — JWT should still be returned	stubSignup(); seedUserIfEmpty throws RuntimeException('timeout')	response.token = JWT
UT-AS-16	signup_neverCallsMatches	signup() should never call passwordEncoder.matches()	stubSignup()	passwordEncoder.matches() never called

7b. login() tests

ID	Method	Scenario	Setup	Expected
UT-AS-17	login_success_completeResponse	Valid credentials login	user found; passwordEncoder.matches(RAW, HASH)=true; JWT generated	response: userId=1; username='alice'; email='alice@example.com'; token=JWT; message contains 'success'
UT-AS-18	login_jwtUsesDbFields	JWT must use the userId and username from the database record	stubLogin()	jwtUtil.generateToken(1L, 'alice') called
UT-AS-19	login_matchesCalledCorrectly	matches() called with raw password and stored hash	stubLogin()	passwordEncoder.matches(RAW, HASH) called
UT-AS-20	login_unknownUsername_throws	Username 'ghost' does not exist	findByUsername('ghost') returns empty	RuntimeException; message contains 'Invalid username or password'
UT-AS-21	login_wrongPassword_throws	Correct username but wrong password	user found; matches('wrong', HASH)=false	RuntimeException; message contains 'Invalid username or password'
UT-AS-22	login_unknownUsername_noToken	Unknown username — no JWT should be generated	findByUsername('ghost') returns empty	jwtUtil.generateToken() never called
UT-AS-23	login_wrongPassword_noToken	Wrong password — no JWT should be generated	user found; matches('bad', HASH)=false	jwtUtil.generateToken() never called
UT-AS-24	login_errorMessages_identical	Error messages for unknown user and wrong password must be identical (no enumeration)	'ghost' not found; 'alice' found but matches returns false for 'bad'	msgA equals msgB
UT-AS-25	login_neverSeedsData	login() should never trigger demo data seeding	stubLogin()	demoDataService has no interactions
UT-AS-26	login_neverModifiesRepo	login() should not modify any user record	stubLogin()	userRepository.save() never called; userRepository.delete() never called

ID	Method	Scenario	Setup	Expected
UT-AS-27	login_findByUsernameOnce	findByUsername() called exactly once during login	stubLogin()	userRepository.findByUsername('alice') called exactly 1 time

8. Auth Controller – MockMvc Tests

File: AuthControllerTest.java | Framework: @WebMvcTest + MockMvc (Spring Security disabled)

8a. POST /api/auth/signup

ID	Method	Input	Expected
CT-AC-01	signup_valid_201	valid JSON: username='alice', email='alice@example.com', password='Secret123!'	HTTP 201; body: token=JWT, userId=1, username='alice', message='User registered successfully'
CT-AC-02	signup_duplicateUsername_400	username already taken — AuthService throws 'Username already exists'	HTTP 400; body.message='Username already exists'
CT-AC-03	signup_duplicateEmail_400	email already registered — AuthService throws 'Email already exists'	HTTP 400; body.message='Email already exists'
CT-AC-04	signup_blankUsername_400	username=''	HTTP 400; AuthService never invoked
CT-AC-05	signup_usernameTooShort_400	username='ab' (2 chars, below 3-char minimum)	HTTP 400; AuthService never invoked
CT-AC-06	signup_usernameTooLong_400	username=21 chars (above 20-char maximum)	HTTP 400; AuthService never invoked
CT-AC-07	signup_invalidEmail_400	email='not-an-email'	HTTP 400; AuthService never invoked
CT-AC-08	signup_blankEmail_400	email=''	HTTP 400; AuthService never invoked
CT-AC-09	signup_passwordTooShort_400	password='abc' (3 chars, below 6-char minimum)	HTTP 400; AuthService never invoked
CT-AC-10	signup_blankPassword_400	password=''	HTTP 400; AuthService never invoked
CT-AC-11	signup_emptyBody_400	body='{}'	HTTP 400
CT-AC-12	signup_error_hasMessageField	AuthService throws RuntimeException	response body has 'message' field of type String

ID	Method	Input	Expected
CT-AC-13	signup_error_noToken	AuthService throws RuntimeException	response body does NOT contain 'token' field

8b. POST /api/auth/login

ID	Method	Input	Expected
CT-AC-14	login_valid_200	valid JSON: username='alice', password='Secret123!'	HTTP 200; body: token=JWT, userId=1, message='Login successful'
CT-AC-15	login_badCredentials_401	wrong password — AuthService throws 'Invalid username or password'	HTTP 401
CT-AC-16	login_badCredentials_bodyHasMessage	wrong password	body.message='Invalid username or password'
CT-AC-17	login_blankUsername_400	username=''	HTTP 400; AuthService never invoked
CT-AC-18	login_blankPassword_400	password=''	HTTP 400; AuthService never invoked
CT-AC-19	login_emptyBody_400	body='{}'	HTTP 400
CT-AC-20	login_error_noToken	AuthService throws on login	response body does NOT contain 'token' field

8c. Security & Edge Cases

ID	Method	Input	Expected
CT-AC-21	signup_usernameWithUnderscore_201	username='alice_123'	HTTP 201
CT-AC-22	signup_xssUsername_201	username='<script>alert(xss)</script>'	HTTP 400 (too long / invalid length)
CT-AC-23	signup_emailPlusAddressing_201	email='alice+test@example.com'	HTTP 201
CT-AC-24	signup_emailSubdomain_201	email='alice@mail.example.com'	HTTP 201
CT-AC-25	signup_veryLongPassword_201	password=100 chars ('A' × 100)	HTTP 201

ID	Method	Input	Expected
CT-AC-26	signup_nullUsername_400	JSON: {username: null, email: 'a@b.com', password: 'Secret123!'}	HTTP 400
CT-AC-27	signup_nullEmail_400	JSON: {username: 'alice', email: null, password: 'Secret123!'}	HTTP 400
CT-AC-28	signup_nullPassword_400	JSON: {username: 'alice', email: 'a@b.com', password: null}	HTTP 400
CT-AC-29	login_sqlInjectionUsername_401	username="admin' OR '1'='1"	HTTP 401
CT-AC-30	login_sqlInjectionPassword_401	password="' OR '1'='1"	HTTP 401
CT-AC-31	login_nullUsername_400	JSON: {username: null, password: 'Secret123!'}	HTTP 400
CT-AC-32	login_nullPassword_400	JSON: {username: 'alice', password: null}	HTTP 400
CT-AC-33	login_whitespaceUsername_400	username=' ' (spaces only)	HTTP 400
CT-AC-34	login_whitespacePassword_400	password=' ' (spaces only)	HTTP 400
CT-AC-35	signup_whitespaceUsername_400	username=' '	HTTP 400
CT-AC-36	signup_whitespaceEmail_400	email=' '	HTTP 400
CT-AC-37	signup_whitespacePassword_400	password=' '	HTTP 400
CT-AC-38	signup_emailNoDomain_400	email='alice@'	HTTP 400
CT-AC-39	signup_emailNoAt_400	email='aliceexample.com'	HTTP 400
CT-AC-40	signup_username3Chars_201	username='abc' (exactly 3 chars — minimum boundary)	HTTP 201
CT-AC-41	signup_username20Chars_201	username='a' × 20 (exactly 20 chars — maximum boundary)	HTTP 201
CT-AC-42	signup_password6Chars_201	password='Pass12' (exactly 6 chars — minimum boundary)	HTTP 201

9. Subscription Detector Service – Unit Tests

File: SubscriptionDetectorServiceTest.java | Framework: JUnit 5 + Mockito (LENIENT)

Detection window: 19–39 days (verified). Deduplication keeps txn if gap from last kept ≥ 20 days.

9 – Grouping

ID	Method	Scenario	Setup	Expected
UT-SD-01	detect_nullCategory_ignored	Transactions with null category are silently skipped	2 expenses with null category, 0 days ago and 30 days ago	returns empty list
UT-SD-02	detect_twoCategories_twoSubscriptions	Two distinct categories both show recurring pattern	2×netflix (30d apart), 2×spotify (30d apart)	returns 2 subscriptions; merchants contain 'netflix' and 'spotify'
UT-SD-03	detect_merchantNameEqualsCategory	Subscription merchant field should equal the transaction category string	2×gym expenses 30d apart	subscription.merchant = 'gym'
UT-SD-04	detect_singleTransactionPerCategory_noDetection	Only one transaction per merchant — no recurring pattern possible	1×netflix expense	returns empty list

9 – Deduplication

ID	Method	Scenario	Setup	Expected
UT-SD-05	detect_exactly20dayGap_passesFilter	Exactly 20-day gap passes the deduplication filter and is detected	2×hulu expenses 20d apart	returns 1 subscription
UT-SD-06	detect_multipleInSameMonth_rejected	Two transactions in same month (within 20 days) — not a subscription	3×spotify: 35d ago, 30d ago, today	returns empty list (gap between first two < 20d after dedup)
UT-SD-07	detect_allTransactionsWithinFilter_noDetection	All transactions within 10 days of each other	3×gym: 10d, 5d, 0d ago	returns empty list

9 – Pattern Window

ID	Method	Scenario	Setup	Expected
UT-SD-08	detect_consecutiveMonths_detected	Transactions in consecutive months (~30d apart) — detected	2×apple expenses 30d apart	returns 1 subscription
UT-SD-09	detect_30days_detected	Exactly 30-day gap — detected	2×netflix expenses 30d apart	returns 1 subscription
UT-SD-10	detect_skippedMonth_notDetected	Transactions 60 days apart (skip a month) — not detected	2×amazon expenses 60d apart	returns empty list
UT-SD-11	detect_sameMonth_notDetected	Transactions only 5 days apart — not a monthly subscription	2×netflix 5d apart	returns empty list
UT-SD-12	detect_noTransactions_returnsEmpty	No transactions at all	empty transaction list	returns empty list
UT-SD-13	detect_singleTransaction_notDetected	Only one transaction in the system	1×netflix	returns empty list

9 – Field Values

ID	Method	Scenario	Setup	Expected
UT-SD-14	detect_avgAmount_isCorrect	avgAmount should reflect the subscription's fixed amount	2×sub expenses both $10.99, 30d apart	subscription.avgAmount = 10.99
UT-SD-15	detect_variableAmounts_rejected	Amounts with >1% variance are rejected as non-subscriptions	3×utilities: $100, $105, $110 each 30d apart	returns empty list
UT-SD-16	detect_lastPaidDate_isLastTransactionDate	lastPaidDate should equal the most recent transaction's date	2×gym 30d apart; last tx date captured	subscription.lastPaidDate = last transaction's date
UT-SD-17	detect_nextDueDate_is30DaysAfterLastPaid	nextDueDate should be lastPaidDate + 30 days	2×netflix 30d apart; last tx is today	subscription.nextDueDate = LocalDate.now().plusDays(30)
UT-SD-18	detect_newSubscription_hasActiveStatus	Newly detected subscription should have ACTIVE status	2×netflix 30d apart; no existing subscription	subscription.status = ACTIVE

ID	Method	Scenario	Setup	Expected
UT-SD-19	detect_subscription_linkedToCorrectUser	Subscription must be linked to the correct user	2×gym 30d apart; userId=1	subscription.user.id = 1

9 – Merging

ID	Method	Scenario	Setup	Expected
UT-SD-20	detect_existingIgnored_statusPreserved	IGNORED subscription re-detected — IGNORED status must be preserved	existing netflix sub with IGNORED status; 2×netflix expenses 30d apart	returned subscription.status = IGNORED
UT-SD-21	detect_existingActive_statusPreserved	ACTIVE subscription re-detected — stays ACTIVE	existing gym sub with ACTIVE status; 2×gym 30d apart	returned subscription.status = ACTIVE
UT-SD-22	detect_existingSubscription_avgAmountUpdated	Existing subscription avgAmount updated to new detected value	existing netflix avgAmount=$5.00; new txns: $20.00 × 2, 30d apart	subscription.avgAmount = 20.00
UT-SD-23	detect_existingSubscription_nextDueDateUpdated	Existing subscription nextDueDate updated to last payment + 30	existing netflix nextDueDate=10d ago; 2×netflix 30d apart	subscription.nextDueDate = LocalDate.now().plusDays(30)
UT-SD-24	detect_newCategoryAlongsideExisting_bothReturned	Existing netflix + new spotify pattern both detected	existing netflix sub; 2×netflix + 2×spotify all 30d apart	returns 2 subscriptions
UT-SD-25	detect_ignoredMerchantWithNoNewTransactions_notReturned	Netflix exists in repo but new transactions are for spotify only	existing netflix sub; 2×spotify 30d apart	result contains only 'spotify'; 'netflix' not in result

9 – Persistence

ID	Method	Scenario	Setup	Expected
UT-SD-26	detect_saveAllCalledWithResults	saveAll() called with the detected subscription list	2×netflix 30d apart	subscriptionRepository.saveAll() called with list of size 1
UT-SD-27	detect_noPatterns_saveAllCalledWithEmpty	No patterns detected — saveAll() still called with empty list	1×netflix (no pattern)	subscriptionRepository.saveAll() called with empty list
UT-SD-28	detect_allTransactionsFetched	Service uses findByUserOrderByTransactionDateDesc() to fetch transactions	empty tx list	transactionRepository.findByUserOrderByTransactionDateDesc(user) called; findByUserAndType never called
UT-SD-29	detect_unknownUserId_throwsAndDoesNotPersist	userId=999 not found — exception thrown, no persistence	userRepository.findById(999) returns empty	RuntimeException thrown; transactionRepository never called; subscriptionRepository.saveAll() never called

9 – findDueSoon()

ID	Method	Scenario	Setup	Expected
UT-SD-30	findDueSoon_delegatesToRepository	findDueSoon() delegates correctly with today..today+7 date range	subscriptionRepository.findDueSoon(user, now, now+7) returns empty	subscriptionRepository.findDueSoon(user, now, now+7) called
UT-SD-31	findDueSoon_returnsRepositoryResult	findDueSoon() returns the repository's list directly	repository returns [activeSub]	result contains exactly the 'due' subscription
UT-SD-32	findDueSoon_dayParameterUsed	days=30 parameter controls the lookahead window	repository stubbed for now..now+30	subscriptionRepository.findDueSoon(user, now, now+30) called
UT-SD-33	findDueSoon_unknownUser_throwsException	userId=999 not found	userRepository.findById(999) returns empty	RuntimeException thrown

10. Subscription Controller – MockMvc Tests

File: SubscriptionControllerTest.java | Framework: @WebMvcTest + MockMvc (Spring Security disabled)

10 – GET /api/subscriptions

ID	Method	Input	Expected
CT-SC-01	valid_200	GET ?userId=1	HTTP 200; body: size=1; merchant='netflix'; status='ACTIVE'
CT-SC-02	noSubscriptions_emptyArray	GET ?userId=1; repo returns empty	HTTP 200; body: empty array []
CT-SC-03	multipleSubscriptions_allReturned	GET ?userId=1; repo returns 3 subs (netflix, spotify, gym-ignored)	HTTP 200; body size=3
CT-SC-04	doesNotCallDetector	GET ?userId=1	HTTP 200; SubscriptionDetectorService has NO interactions
CT-SC-05	missingUserId_400	GET (no userId param)	HTTP 4xx
CT-SC-06	dtoFieldsComplete	GET ?userId=1; repo returns 1 subscription	body[0] has: id, merchant, avgAmount, status, lastPaidDate, nextDueDate fields

10 – POST /api/subscriptions/detect

ID	Method	Input	Expected
CT-SC-07	valid_200	POST ?userId=1; detectorService returns [netflix]	HTTP 200; body size=1; merchant='netflix'
CT-SC-08	noDetections_emptyArray	POST ?userId=1; detectorService returns empty	HTTP 200; body: empty array []
CT-SC-09	missingUserId_400	POST (no userId param)	HTTP 4xx
CT-SC-10	callsServiceWithCorrectUserId	POST ?userId=42	detectorService.detectSubscriptions(42L) called
CT-SC-11	ignoredInResult_statusReflected	POST ?userId=1; detectorService returns gym sub with IGNORED status	body[0].status='IGNORED'

10 – GET /api/subscriptions/due-soon

ID	Method	Input	Expected
CT-SC-12	default7Days_200	GET ?userId=1; repo returns netflix due in 2 days	HTTP 200; body size=1; merchant='netflix'

ID	Method	Input	Expected
CT-SC-13	nothingDue_emptyArray	GET ?userId=1; repo returns empty	HTTP 200; body: empty array []
CT-SC-14	customDays_accepted	GET ?userId=1&days=14	HTTP 200; subscriptionRepository.findDueSoonByUserId called
CT-SC-15	missingUserId_400	GET (no userId param)	HTTP 4xx
CT-SC-16	activeReturned	GET ?userId=1; repo returns active sub	body[0].status='ACTIVE'

10 – PUT /api/subscriptions/{id}/ignore

ID	Method	Input	Expected
CT-SC-17	valid_200_withIgnoredStatus	PUT /subscriptions/1/ignore?userId=1; sub found	HTTP 200; body.status='IGNORED'; body.merchant='netflix'
CT-SC-18	setsIgnoredBeforeSave	PUT /subscriptions/1/ignore?userId=1	subscriptionRepository.save() called with entity where status=IGNORED
CT-SC-19	savesExactlyOnce	PUT /subscriptions/1/ignore?userId=1	subscriptionRepository.save() called exactly once
CT-SC-20	missingUserId_400	PUT /subscriptions/1/ignore (no userId param)	HTTP 4xx
CT-SC-21	alreadyIgnored_idempotent_200	PUT /subscriptions/1/ignore?userId=1; sub already IGNORED	HTTP 200; body.status='IGNORED'

11. Subscription Controller (Create) – MockMvc Tests

File: SubscriptionControllerCreateTest.java | Framework: @WebMvcTest + MockMvc

11 – Valid Requests

ID	Method	Input	Expected
CT-SCC-01	create_valid30Days_200	POST ?userId=1; merchant='Netflix', amount=15.99, lastPaid=today-5, nextDue=today+25	HTTP 200; body.merchant='Netflix'; body.avgAmount=15.99

ID	Method	Input	Expected
CT-SCC-02	create_exactly25Days_200	25-day gap (minimum boundary)	HTTP 200
CT-SCC-03	create_exactly35Days_200	35-day gap (maximum boundary)	HTTP 200
CT-SCC-04	create_lastPaidToday_200	lastPaidDate = today	HTTP 200
CT-SCC-05	create_nextDueToday_200	nextDueDate = today	HTTP 200

11 – Date Range Validation

ID	Method	Input	Expected
CT-SCC-06	create_24DaysGap_400	24-day gap (below 25-day minimum)	HTTP 400; body.message='Next due date must be at least 25 days after last paid date'
CT-SCC-07	create_36DaysGap_400	36-day gap (above 35-day maximum)	HTTP 400; body.message='Next due date must be at most 35 days after last paid date'
CT-SCC-08	create_nextDueBeforeLastPaid_400	nextDueDate is 1 day before lastPaidDate	HTTP 400

11 – Field Validation

ID	Method	Input	Expected
CT-SCC-09	create_blankMerchant_400	merchant=''	HTTP 400; subRepo never called
CT-SCC-10	create_merchantTooLong_400	merchant = 'A' × 101 (above 100-char limit)	HTTP 400; subRepo never called
CT-SCC-11	create_amountZero_400	amount=0.00	HTTP 400; subRepo never called
CT-SCC-12	create_amountNegative_400	amount=-10.00	HTTP 400; subRepo never called
CT-SCC-13	create_amountTooHigh_400	amount=10000.01 (above $10,000 limit)	HTTP 400; subRepo never called
CT-SCC-14	create_lastPaidFuture_400	lastPaidDate = tomorrow	HTTP 400; subRepo never called

ID	Method	Input	Expected
CT-SCC-15	create_nextDuePast_400	nextDueDate = yesterday	HTTP 400; subRepo never called
CT-SCC-16	create_missingUserId_400	no userId param	HTTP 400
CT-SCC-17	create_nullMerchant_400	merchant=null	HTTP 400
CT-SCC-18	create_nullAmount_400	no amount field in JSON	HTTP 400
CT-SCC-19	create_emptyBody_400	body='{}'	HTTP 400

12. Summary

Test File	Category	# Tests
FraudDetectionServiceTest	Unit	6
FraudDetectionIntegrationTest	Integration	11
FraudDetectionServicePropertyTest	Property-Based	4
FraudAlertServiceTest	Unit	11
TransactionServiceTest	Unit	7
DemoDataServiceTest	Unit	10
AuthServiceTest (signup)	Unit	16
AuthServiceTest (login)	Unit	11
AuthControllerTest (signup)	Controller/MockMvc	13
AuthControllerTest (login)	Controller/MockMvc	7
AuthControllerTest (security)	Controller/MockMvc	22
SubscriptionDetectorServiceTest	Unit	33
SubscriptionControllerTest	Controller/MockMvc	21
SubscriptionControllerCreateTest	Controller/MockMvc	19
TOTAL		**191**